Insurance Suite

Implementation document: a Guidewire-style policy, billing and claims system

How to build a core insurance system that works the way Guidewire InsuranceSuite works: policy administration, billing, claims and a shared contact manager, driven by configuration rather than code. It starts from what the PolicyCenter Simulator already does and describes what to build, in what order, and how.

Version	1.0, draft for review
Date	18 October 2026
Starting point	PolicyCenter Simulator (github.com/Sreenu123196/Policy_Center), as described in its project explanation. The code itself was not reviewed.
First market	India, with the design kept market-neutral so US states can be added from the simulator's rate books
Recommended stack	TypeScript on Node.js, PostgreSQL (section 3)

> **Independent design.** This document follows the publicly known concepts of Guidewire's products (accounts, jobs, policy periods, effective dating, product model, rating, underwriting authority, billing and claims structures). It does not use Guidewire's code, data model files or documentation, and the built system must not use Guidewire's names or branding. "Guidewire", "PolicyCenter", "BillingCenter" and "ClaimCenter" are Guidewire Software trademarks, so the suite needs its own product names before it goes to real customers.

Contents

1. Purpose, scope and assumptions
2. Solution overview
3. Technology stack
4. Core concepts
5. Policy data model
6. Effective dating and out-of-sequence changes
7. Product model
8. Jobs and their life cycles
9. Rating engine
10. Underwriting and authority
11. Forms and documents
12. Activities, queues and workflow
13. Security and administration
14. Billing module
15. Claims module
16. Contact manager
17. APIs, events and integrations
18. Configuration management
19. Non-functional requirements
20. Screens
21. Testing strategy
22. Delivery plan
23. From the simulator to the suite
24. Open decisions and risks
25. Appendices: schema, algorithm, examples, glossary

1. Purpose, scope and assumptions

Purpose

Give the team one reference for building a production-grade core insurance system with Guidewire-style depth: every policy change kept as a dated version, products and prices defined as data, underwriting authority by person, and billing and claims connected to the policy.

In scope

Module	What it covers
Policy (PolicyCenter-style)	Accounts, products, quoting, binding, issuance, policy changes including out-of-sequence ones, cancellation, reinstatement, renewal, rewrite, premium audit, rating, underwriting, forms and documents.
Billing (BillingCenter-style)	Charges, payment plans, invoices, payments, delinquency, refunds, producer commissions, general-ledger export.
Claims (ClaimCenter-style)	First notice of loss, coverage verification, exposures, reserves, payments, recoveries, surveyors and litigation.
Contacts (address-book style)	One shared record for every person and company, with KYC status, identifiers and consent.
Platform	Security, activities and queues, workflow, documents, reference data, audit history, events, batch jobs, APIs, admin screens.

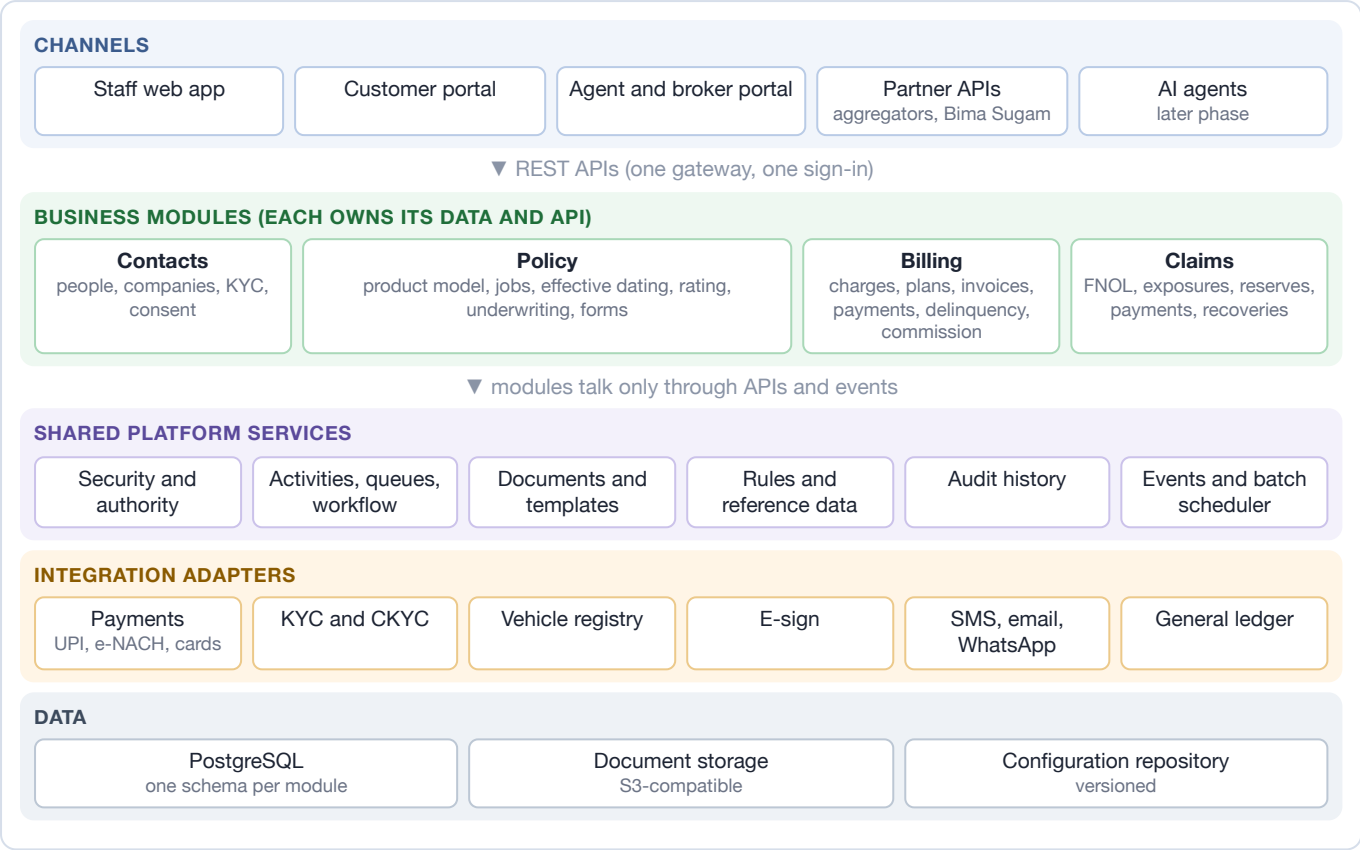
Out of scope for version 1

- Developing actuarial rates. Actuaries supply the rates; the suite stores and applies them.
- Reinsurance accounting, the full general ledger and a data warehouse. The suite exports to these systems.
- The AI agent layer. It plugs in after the core is stable, as described in the separate "Autonomous operations deep dive (v2)".

Assumptions

- **First market: India.** First lines: private car and two-wheeler motor, then home and small-business packages. Other lines and US states come from the simulator's eight US lines.
- **Team:** about 6 to 8 people (product owner, insurance analyst, 3 to 4 developers, tester, DevOps engineer), with actuarial and legal help on call.
- The simulator is the behavioural reference: its workflows and 132 tests describe what the first version must do at minimum.

2. Solution overview



Architecture style: a modular monolith first. All modules are deployed together at the start, but each has its own database schema, its own API and its own events, and no module reads another's tables. This keeps the first releases simple and lets modules be split into separate services later without a rewrite. Billing and Claims connect to Policy exactly as Guidewire's separate products do: through APIs and messages.

3. Technology stack

Need	Recommended	Why
Language and runtime	TypeScript on Node.js (LTS)	Same platform as the simulator, so its knowledge and tests carry over; types catch errors in a large domain model.
Database	PostgreSQL	Date-range types and exclusion constraints are well suited to effective dating (section 6); strong transactions.
Data access	SQL-first query builder (for example Kysely)	Effective-dated queries need exact control over SQL.
Long-running processes	A workflow engine (Temporal) or a PostgreSQL job queue (pg-boss)	Renewals, delinquency and claim follow-ups run for weeks, with timers and retries.
Rules	Rules as data, evaluated by a small engine (for example json-rules-engine or decision tables)	Business users change rules without a code release.
Documents	HTML templates rendered to PDF by headless Chromium	Easy to localise into English, Hindi and Telugu.

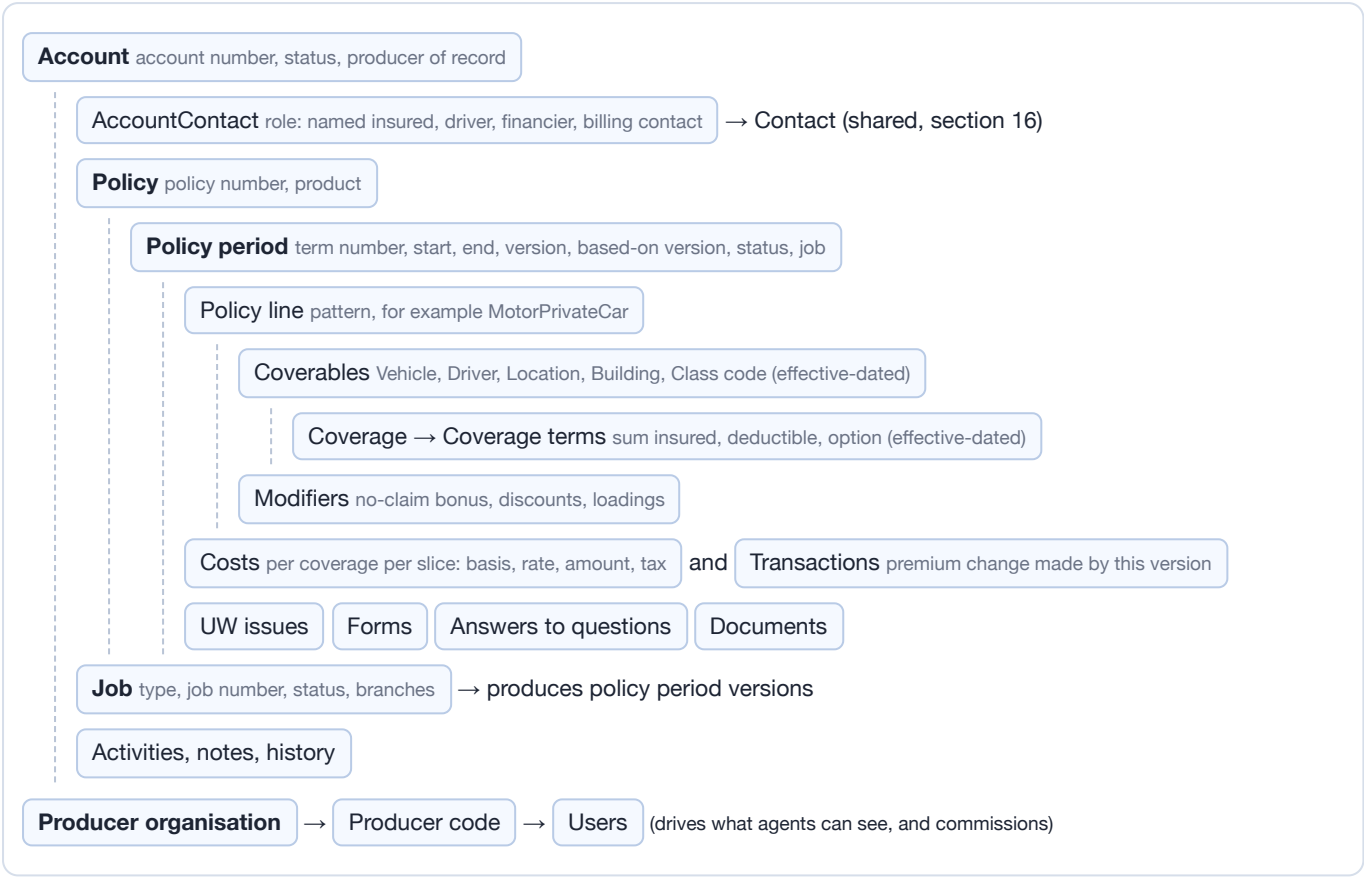
Need	Recommended	Why
Sign-in	OpenID Connect (for example Keycloak) with MFA	Single sign-on, MFA, and separate realms for staff, agents and customers.
User interface	React with TypeScript	Large forms and wizards; shared types with the API.
APIs	REST with OpenAPI 3 contracts	Partners and AI agents use the same documented API.
Observability	OpenTelemetry, metrics and central logs	Trace a request through every module.
Hosting	Containers on a cloud region in India (for example AWS Mumbai or Hyderabad, Azure Pune or Chennai, Google Cloud Mumbai or Delhi)	Keeps policy and claim records in India; confirm the current IRDAI record-keeping rule.

Alternative: Java or Kotlin with Spring Boot, closer to Guidewire's own JVM world. The design in this document works the same either way.

4. Core concepts

Concept	Meaning in this suite
Account	The customer relationship. It holds contacts, policies, the billing account and claims.
Policy	The contract across all its terms. It has a policy number that never changes.
Policy period	One version of one term (for example 1 April 2026 to 1 April 2027, version 3). Every bound job creates a new version.
Job	A unit of work that changes a policy: Submission, Issuance, Policy Change, Cancellation, Reinstatement, Renewal, Rewrite, Audit.
Branch	A working copy of a policy period inside a job. A submission can have several branches (quote options); one is bound.
Policy line	The type of cover inside a product, for example Motor Private Car or Home Building and Contents.
Coverable	The thing insured: a vehicle, driver, location, building or business class.
Coverage, term	A cover (for example Own Damage) and its values (sum insured, deductible, options).
Effective-dated	Data that is true only between two dates inside the period. The system can show the policy as it was on any date.
Slice	A stretch of the period in which nothing changes. Each slice is rated separately and prorated.
Rate book	The versioned set of rate tables and rating steps for a product, a jurisdiction and a date range.
UW issue, authority profile	A risk question raised by a rule, and the record of who may approve which issues up to which values.
Activity	A task for a person or queue, with a due date and escalation.
Typelist	A managed list of codes (cancellation reasons, loss causes, states) with names in each language.

5. Policy data model



Rule	Detail
Stable identity	Every effective-dated object has a *fixed id* that stays the same across slices and versions, plus a row id per slice. "Vehicle 1" is the same fixed id in every version.
Versions are copies	A new job copies the latest bound version into a new branch. Bound versions are never edited again.
Money	Stored as integers in the smallest unit (paise or cents) with a currency code; rounding rules come from the product.
Costs and transactions	Costs say what the version charges. Transactions say what changed compared with the version it was based on. Billing receives transactions.
Audit	Every change records who made it (person or agent), when, and through which job.

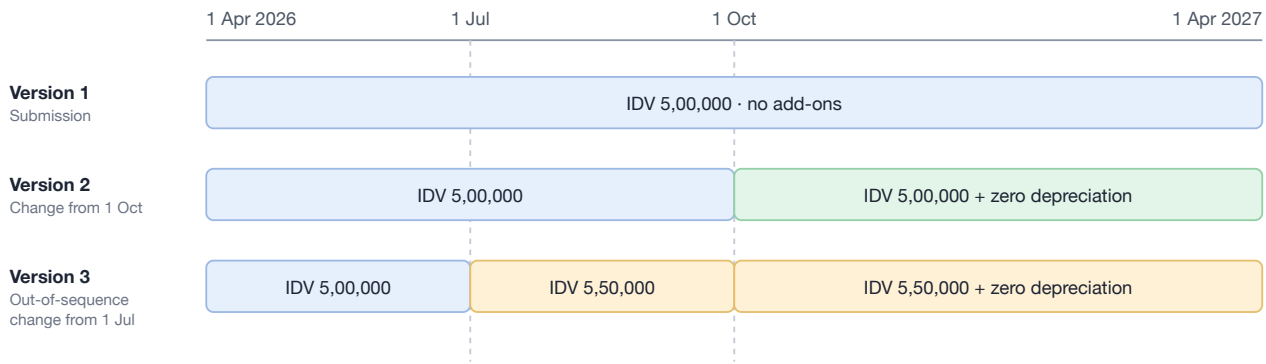
6. Effective dating and out-of-sequence changes

This is the most important technique in a Guidewire-style system. Policies change during their term, so the system must answer two questions at any time: *what did the policy look like on a given date*, and *what does each stretch of time cost*.

How it works

1. Every effective-dated row has `eff_from` and `eff_to` inside the policy period, plus its branch and fixed id.
2. A user editing a policy chooses an **edit effective date**. The system splits any row that spans that date and changes only the part from that date to the end of the period (the *window*).
3. For rating, the period is cut at every date where anything changes. Each **slice** is rated and prorated by days.

4. Binding the job saves the branch as a new version. Its transactions are the difference between its costs and the costs of the version it was based on.



Out-of-sequence changes

A change is **out of sequence** when its effective date is earlier than a change that is already bound. The system must apply it without losing the later change:

1. Start from the latest bound version (version 2 above).
2. Apply the new change from its effective date to the end of the period.
3. For each later bound change, check every field the new change touched. If the later change set the same field, there is a **conflict**: the later change wins from its own date by default, and the user is shown each conflict to confirm or override.
4. Re-rate every slice and create transactions against version 2.

Preemption. Two jobs can be open on the same policy at once. When one binds, the other is *preempted*: before it can bind, it must be rebased onto the new version, using the same merge as an out-of-sequence change. Optimistic locking (a version number on the policy period) stops two binds from colliding.

The database enforces that slices of one object never overlap (Appendix A), and the merge algorithm is in Appendix B.

7. Product model

Products are **configuration, not code**. Adding a coverage or a state should need a configuration release, not a programming change.

Element	What it defines
Product	Name, code, policy lines, allowed terms (annual, multi-year), jurisdictions, and the version in force by date.
Policy line pattern	Which coverables the line has (vehicle, driver, location) and which coverages each can carry.
Coverage pattern	Code, name in each language, category, and *existence*: required, electable (optional) or suggested.
Term pattern	Each value of a coverage: a list of options, a direct value with minimum and maximum, or a package of linked values; plus the default.
Exclusions and conditions	Clauses that attach to the policy, with the same availability rules.
Availability rules	When a pattern can be offered: jurisdiction, date range, product version, and expressions such as "vehicle age 5 years or less".

Element	What it defines
Dependencies	"A requires B" and "A excludes B" between coverages.
Question sets	Underwriting and eligibility questions, with answers feeding rules and rating.
Coverage terms of different length	One policy can hold covers with different terms, for example a 3-year third-party cover with a 1-year own-damage cover for a new car in India.

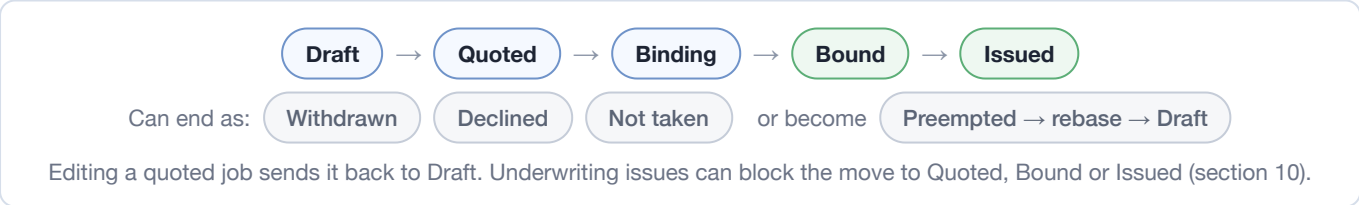
The product model is stored as versioned JSON in the configuration repository (section 18), checked against a schema, and loaded at start-up. New business uses the product version in force on the period start date; renewals move to the newest version under rules set per product. An example is in Appendix C.

Example product set for India

Product	Notes for configuration
Private car and two-wheeler	Third-party liability (required by law, at rates set by the authorities), own damage on the Insured Declared Value (IDV), compulsory personal accident cover for the owner-driver, no-claim bonus, add-ons such as zero depreciation, engine protection and roadside assistance; long-term third-party cover for new vehicles.
Home	The IRDAI standard home product (Bharat Griha Raksha) and optional covers.
Small business	The standard packages for micro and small enterprises (Bharat Sookshma and Bharat Laghu Udyam Suraksha).
Employee's compensation, liability, commercial vehicle	Later phases; employee's compensation needs premium audit on wages.

Confirm each product's current rules and filings with the insurer's product and compliance teams; the suite only stores what they approve.

8. Jobs and their life cycles



Editing a quoted job sends it back to Draft. Underwriting issues can block the move to Quoted, Bound or Issued (section 10).

Job	Purpose and main steps	Key rules
Submission	New business. Quick quote or full application; several quote versions side by side; bind and issue.	Product version and rate book chosen by start date; quote valid for a set number of days.
Issuance	Issues a policy that was bound earlier with missing details.	Only after a bind-only submission.
Policy change	Mid-term change, for example adding a vehicle or changing an address.	Effective date inside the period; out-of-sequence allowed; premium difference becomes a charge or refund.
Cancellation	Ends cover before expiry: now or on a future date.	Reason code and source (insurer or insured); refund method *flat* (from start), *pro rata*, or *short rate* (from a table); notice days by jurisdiction and reason.
Reinstatement	Restores a cancelled policy.	With or without a gap in cover; may need underwriting and a fee.

Job	Purpose and main steps	Key rules
Renewal	Creates the next term.	Starts a set number of days before expiry; pre-renewal direction (renew, non-renew, review); re-rated with the rate book for the new term; renewal offer, then accepted or not taken; non-renewal needs approval and a notice.
Rewrite	Replaces a policy when a change cannot be made as an endorsement (for example a product change).	Types: full term, remainder of term, new account.
Audit	Premium audit for lines rated on wages or turnover.	Final audit after expiry (optionally preliminary or checking audits); reprice on actual figures; charge or refund the difference.

Refund formulas: pro rata refund = term premium × unexpired days ÷ term days. Short-rate refund = term premium × (1 − short-rate factor for the days in force). Flat = full refund. Minimum retained premium and fees come from the product.

9. Rating engine

Part	What it is
Rate book	All rating data for one product, jurisdiction and date range. Status: Draft → In review → Approved → Active → Retired. Only Active books are used for live quotes.
Rate tables	Lookup tables: keys (exact values or ranges) to factors, for example zone and engine size to base rate, or claim-free years to no-claim bonus.
Rate routines	Ordered calculation steps in a small declarative format: look up, multiply, add, apply minimum, round. No free-form code.
Worksheet	Every step recorded with its input, table, factor and result. Shown to underwriters and stored with the version.
Impact testing	Run a candidate rate book against a sample of policies in force and compare premiums before approval.

Rating flow

1. Build the slices of the period (section 6).
2. Choose the rate book: product, jurisdiction, and the period start date (renewals use the new term's date).
3. For each slice and coverage: compute the basis (for example the IDV), run the routine, and produce a cost with its worksheet.
4. Prorate each cost by the slice's days; add taxes (for example GST) and fees as separate cost items; apply minimum premiums and rounding.
5. Compare with the previous version to create transactions.

Rating is a pure function: the same policy snapshot and rate book always give the same result. That makes it easy to test, to replay old quotes, and to run impact tests. Active rate books are cached in memory. An example routine is in Appendix D.

10. Underwriting and authority

Part	Design
UW rules	Conditions checked at fixed points: before quote, before bind, before issuance, before renewal. Each raises an issue of a given type with a value, for example "driver age 19" or "IDV 45,00,000".
Issue types	Code, description, the point it blocks (quote, bind, issuance, or non-blocking), how long an approval lasts, and how values are compared.
Authority profiles	Sets of grants: which issue types a person may approve, up to which value. Users get profiles by role or by name.
Referral	If the user lacks authority, "Request approval" creates an activity for an underwriter queue with the full context.
Approval	Approve (optionally up to a value and until a date), reject, or approve with conditions. Every decision is kept with who decided and why.
Re-checking	After any edit the rules run again. An existing approval still counts if the new value stays inside what was approved; otherwise the issue reopens.

11. Forms and documents

- **Form patterns:** form code, edition date, lines and jurisdictions, and an *inference rule* that says when the form is attached (a coverage was chosen, a jurisdiction, a job type). On each change, forms are re-inferred, added, removed or replaced.
- **Documents produced:** proposal form, policy schedule, motor certificate of insurance, endorsement schedule, renewal notice, cancellation notice, claim letters, invoices and receipts.
- **Templates** in HTML with placeholders, one per language, rendered to PDF and stored with metadata (account, policy, job, version, form codes).
- **E-signature** for proposals and declarations (for example Aadhaar-based e-sign in India); **delivery** by email, WhatsApp or the portal, according to the customer's consent.

12. Activities, queues and workflow

Part	Design
Activity patterns	Templates for tasks: subject, priority, target days, escalation days, default queue.
Queues and assignment	Groups and queues; assignment rules by group, region, line, language, workload or round robin.
Escalation	A batch job raises the priority or reassigns overdue activities and alerts the supervisor.
Notes and history	Notes with a security level; an event history on every account, policy, billing account and claim.
Workflow	Long-running processes (renewal, delinquency, claim follow-ups) in the workflow engine, with timers on business days from a holiday calendar.

The simulator's tickets (with deadlines of 4, 24, 48 and 120 hours) become one use of activities and queues.

13. Security and administration

- **Users, groups and roles:** groups in a hierarchy (region → branch → team); roles hold permissions such as "view account", "create submission", "bind", "approve UW issue", "view billing", "make claim payment".

- **Producer code security:** agents and brokers see only accounts and policies under their producer codes, as the simulator does today by agency.
- **Authority:** underwriting authority profiles (section 10) and claim financial limits (section 15).
- **Separation of duties:** the person who writes a rate book, a limit or a rule cannot approve it.
- **Admin screens:** users, groups, roles, authority profiles, activity patterns, holiday calendars, typelists, product model, rate books, forms, payment and delinquency plans. Every admin change goes to the audit log.
- **Sign-in:** single sign-on with MFA for staff; separate sign-in for agents and customers; sessions that survive restarts; HTTPS everywhere.

14. Billing module

Concept	Design
Billing account	Usually one per policy account; holds policy periods, invoices, payments and the payment instrument.
Charges	Premium, taxes and fees received from Policy per transaction, split into invoice items by the payment plan.
Payment plans	Full pay, two-pay, quarterly, monthly; down payment percentage; instalment fees.
Billing plans	Invoice lead days, minimum invoice amount, fees, grace days.
Invoices	Generated by a batch job on the invoice date; tax-invoice fields as GST rules require.
Payments	UPI (including AutoPay mandates), e-NACH, cards, bank transfer; allocation rules (oldest first, by policy); suspense for unmatched money.
Delinquency plans	Timed steps after a missed payment: reminder, final notice, then a *cancellation request* sent to Policy; reinstatement if paid later.
Refunds	Return premium from changes and cancellations, paid out or applied to other amounts due.
Commissions	Commission plans by producer, line and tier; earned when billed or when paid; statements and payouts.
Accounting	Double-entry sub-ledger for every money movement; daily export to the general ledger.

How Policy and Billing talk: Policy sends a billing instruction on bind, change, cancellation, reinstatement, renewal and audit. Billing sends back payment status, delinquency cancellation requests and reinstatement triggers. Both sides are idempotent: the same message twice has no extra effect.

15. Claims module

Concept	Design
FNOL	First notice of loss from staff, portal, phone, WhatsApp or partners: loss date and time, cause, place, description, photos.
Policy as of the loss date	Claims asks Policy for the policy *as it was on the loss date*, using effective dating: in force or not, which vehicle, which covers, which deductibles.
Incidents and exposures	Incidents describe damage or injury; exposures are one claimant against one coverage, each with its own status and handler.
Reserves	Reserve lines by exposure, cost type (claim cost or expense) and category; history of every change.

Concept	Design
Payments	Partial or final; payee checks; bank transfer or UPI; approval limits by person (claim financial authority).
Recoveries	Subrogation, salvage and deductible recoveries, booked against reserves.
Handling	Segmentation (fast track or complex), assignment, activities, surveyor appointment where the rules require it, cashless garage network, fraud indicators, litigation, close and reopen.

Financial view: incurred = paid + outstanding reserves – recoveries. The simulator's claim stages (Submitted, Under Review, Documents Pending, Approved, Settled, Rejected) become claim and exposure statuses.

16. Contact manager

- One record per person or company, shared by Policy, Billing and Claims through its id.
- Names, addresses (with PIN code validation), phones, emails, date of birth, and identifiers such as PAN, GSTIN and the CKYC number. Aadhaar is never stored in full; it is masked or tokenised.
- KYC status and evidence, and consent per channel (call, SMS, email, WhatsApp) with dates.
- Duplicate matching rules (name, date of birth, phone, identifiers) and a merge process.
- Roles on accounts and claims (named insured, driver, financier, claimant, witness, garage, surveyor).

17. APIs, events and integrations

APIs

- REST, resource-based, versioned (`/v1/...`) and documented in OpenAPI.
- Business actions are explicit endpoints, for example `POST /v1/jobs/{id}/quote`, `/bind`, `/issue`, `/withdraw`.
- Every POST takes an idempotency key; updates carry a version (ETag) to stop lost updates; lists are paged and filtered.
- The same API serves the web app, portals, partners and, later, AI agents, with the same permissions.

Events

Each module writes its events to an **outbox** table in the same transaction as the change, then publishes them in order per account. Consumers are idempotent. Main events: `PolicyBound`, `PolicyChanged`, `PolicyCancelled`, `PolicyRenewed`, `ChargesCreated`, `PaymentReceived`, `DelinquencyStarted`, `ClaimOpened`, `ReserveChanged`, `ClaimPaymentIssued`.

Integration adapters

Each outside service sits behind an adapter with a fixed interface, so suppliers can change without touching business code: payment gateway, KYC and CKYC, vehicle registration lookup (for example VAHAN, where access is permitted), PIN code and address validation, e-sign, SMS, email and WhatsApp, fraud data, the general ledger and reinsurance.

Batch jobs

Renewal start, policy expiry, invoice generation, delinquency steps, activity escalation, commission runs, and archiving. A scheduler keeps the run history and can re-run any job safely.

18. Configuration management

- Everything that changes with the business lives in versioned configuration: typelists, product model, rate books, UW rules and issue types, forms, activity patterns, payment, billing and delinquency plans, document

templates.

- Configuration is stored in a repository as files, edited through admin screens by business users, and checked against schemas.
- It is promoted from development to test to production through a pipeline with approvals, and each version carries the dates from which it applies.
- Typelists hold a code, names in English, Hindi and Telugu, a sort order and a retired flag. Codes are never reused.

19. Non-functional requirements

Area	Target for version 1
Availability	99.9% for staff and portal hours; planned maintenance outside business hours
Speed	Quote for personal lines under 3 seconds; screen loads under 2 seconds
Scale	Designed for 1 million policies and 100 concurrent staff users at first, with room to grow
Backup and recovery	Lose at most 15 minutes of data; restore within 4 hours; restore tested every quarter
Security	OWASP ASVS level 2; encryption in transit and at rest; secrets in a vault; yearly penetration test
Privacy	DPDP Act: notice, consent, purpose limits, retention schedules, breach reporting
Data location	Policy and claim records held in India; confirm the current IRDAI rule
Audit	Every change kept with who, when and why, for the legal retention period
Languages and formats	English, Hindi and Telugu; Indian number format (lakh, crore); dates as DD-MM-YYYY
Accessibility	WCAG 2.2 level AA

20. Screens

Area	Screens
Desktop	My activities, my queues, my submissions, alerts
Search	Accounts, policies, contacts, claims, billing accounts
Account file	Summary, contacts, policies, billing summary, claims, documents, notes, history
Policy file	Summary, version timeline, coverages as of any date, transactions, forms, documents, UW issues, history
Job wizard (per line)	Policy information → coverables (vehicles, drivers or locations) → coverages → questions → risk analysis (UW issues) → quote (versions side by side, with worksheet) → forms → payment → bind and issue
Billing	Billing account summary, invoices, payments, delinquency, commissions
Claims	FNOL wizard, claim summary, exposures, reserves, payments, recoveries, activities
Admin	Users and security, product model, rate books, rules, forms, plans, typelists, batch jobs

21. Testing strategy

Test type	What it proves
Rating golden tests	Fixed policies with expected worksheets for every rate book; any change in premium is visible in review.
Effective-dating scenarios	A matrix of change dates: in sequence, out of sequence, same-day changes, preemption, cancellation and reinstatement after changes.
Job state machine	Every allowed and forbidden move for every job type.
Permissions matrix	Every role against every action, through the API (as the simulator's role tests do today).
Billing and claims money	Allocation, refunds, commissions, reserves and payments always balance in the sub-ledger.
Contract tests	APIs and events match their published schemas between modules.
End-to-end journeys	Quote to bind to change to claim to renewal, in each language.
Performance and security	Load at the targets in section 19; automated security scans; penetration test before go-live.
User acceptance	Business scenarios signed off by underwriting, billing and claims leads; parallel run against the old system if migrating data.

22. Delivery plan

Rough durations for the team in section 1. Phases 3 and 4 can overlap once Phase 2 is stable.

Phase 0 - Foundations (about 2 months). Platform, sign-in and security model, contact manager, configuration framework and typelists, audit history, events and outbox, CI/CD, hosting in India.

Exit: a user can sign in, create a contact and an account; every change is audited; configuration can be promoted between environments.

Phase 1 - Policy core (about 3 to 4 months). Product model, effective-dated data, submission with multiple quote versions, rating engine with rate book life cycle and worksheets, UW rules and authority, forms and documents. First product: private car.

Exit: a private car policy can be quoted, referred, approved, bound and issued with correct documents; rating golden tests pass.

Phase 2 - Policy life cycle (about 3 months). Policy change including out-of-sequence, preemption, cancellation, reinstatement, renewal, rewrite; two-wheeler product.

Exit: the effective-dating scenario matrix passes in full.

Phase 3 - Billing (about 3 months). Charges, payment plans, invoices, UPI and e-NACH payments, delinquency with cancellation requests, refunds, commissions, GL export.

Exit: a year of simulated billing activity balances to the paisa.

Phase 4 - Claims (about 3 to 4 months). FNOL, policy as of loss date, exposures, reserves, payments, recoveries, surveyors, activities.

Exit: motor own-damage and third-party claims can be handled end to end with correct financials.

Phase 5 - More lines and go-live readiness (about 3 months). Home and small-business packages, premium audit, performance and security testing, data migration, compliance review, parallel run.

Exit: section 19 targets met; sign-off from business, security and compliance.

Phase 6 - AI agent layer (after go-live). As described in the "Autonomous operations deep dive (v2)", built on the same APIs and permissions.

Exit: agents promoted level by level on measured results.

Total: about 15 to 18 months from start to go-live for motor, home and small-business lines across policy, billing and claims.

23. From the simulator to the suite

Simulator today	Action	In the suite
`rating.js`, `rateBooks.js`	Evolve	Rating engine with rate tables, routines, worksheets and rate book life cycle; keep the factor-worksheet idea
`uwRules.js` (authority levels)	Evolve	UW rules, issue types with blocking points, authority profiles and grants
`productModel.js`, `validation.js`	Evolve	Product model as versioned configuration with availability and dependency rules
Jobs in `policyCenter.js`	Extend	All eight job types, branches, effective dating, out-of-sequence and preemption
`store.js` (one JSON file)	Rebuild	PostgreSQL with effective-dated tables, outbox and optimistic locking
`permissions.js`, `auth.js`	Evolve	Roles and permissions, producer code security, single sign-on with MFA
Billing inside policies; `finance.js`	Rebuild	Billing module with plans, invoices, payments, delinquency, commissions and sub-ledger
Claim stages	Rebuild	Claims module with exposures, reserves, payments and recoveries
Tickets with deadlines	Evolve	Activities, queues, assignment and escalation
`messaging.js`	Evolve	Communications service with templates per language, consent and delivery tracking
`chatbot.js`, `speech.js`	Keep for later	The AI agent layer (Phase 6)
132 tests	Keep	Behaviour reference; rewritten against the new API as acceptance tests

24. Open decisions and risks

Decision or risk	Notes
Build or buy	Guidewire's products reflect many years of work by a very large team. Building a focused suite for a few lines is realistic; matching all of Guidewire's breadth is not a 15-month goal. Decide the product scope first and keep it narrow.
Who will run it	An insurer, a broker or a demo and training product? This decides licences, the compliance work and how much of billing and claims is needed.
Product names	New names for the suite and modules, free of Guidewire trademarks.
Stack	TypeScript and Node.js (recommended) or Java and Spring Boot. Decide before Phase 0.

Decision or risk	Notes
Data location	Confirm the current IRDAI record-keeping rule and choose the cloud region.
Complexity of effective dating	The biggest technical risk. Build the scenario test matrix in Phase 1, before policy change work starts.
Rates and products	Need approved products and rates from actuarial and compliance teams; the suite cannot invent them.
Data migration	If replacing an existing system, plan conversion and a parallel run early.

Appendix A. Effective-dated table (PostgreSQL)

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE policy.coverage (
  row_id        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  branch_id     bigint NOT NULL REFERENCES policy.policy_period(id),
  fixed_id      uuid   NOT NULL,          -- same logical coverage in every slice and version
  coverable_fid uuid   NOT NULL,          -- fixed id of the vehicle, location ...
  pattern_code  text   NOT NULL,          -- e.g. 'MOTOR_OD', 'ZERO_DEP'
  eff_from      date   NOT NULL,
  eff_to        date   NOT NULL,          -- exclusive
  terms         jsonb  NOT NULL,          -- e.g. {"idv": 55000000, "vol_deductible": 250000}
  created_by    text   NOT NULL,          -- user or agent id
  created_at    timestamptz NOT NULL DEFAULT now(),
  CHECK (eff_from < eff_to),
  -- slices of one coverage in one branch may never overlap
  EXCLUDE USING gist (branch_id WITH =, fixed_id WITH =,
                      daterange(eff_from, eff_to) WITH &&)
);

-- The policy as of a date: one row per coverage
SELECT * FROM policy.coverage
 WHERE branch_id = $1 AND eff_from <= $2 AND $2 < eff_to;
```

Appendix B. Out-of-sequence merge (pseudocode)

```
function applyOutOfSequence(policy, newJob):
  base   = latestBoundVersion(policy)                // e.g. version 2
  branch = copyBranch(base)                          // new working copy
  d      = newJob.effectiveDate                      // e.g. 1 July
  later  = boundChangesAfter(policy, d)              // e.g. the 1 Oct change

  for each edit in newJob.edits:                     // field-level edits
    splitRowsAt(branch, edit.object, d)              // cut the slice at d
    for each slice of edit.object from d to periodEnd:
      laterChange = later.find(c => c.changed(edit.object, edit.field)
                                 and c.date <= slice.from)
      if laterChange is none:
        slice.set(edit.field, edit.value)            // carry forward
      else:
        conflicts.add({edit, laterChange, slice})    // later change wins by default
  if conflicts not empty:
    showConflicts(conflicts)                         // user confirms or overrides
  rateAllSlices(branch)
  branch.transactions = diffCosts(branch, base)      // charge or refund
  return branch
```

Appendix C. Product model (example)

```
{
  "product": "MOTOR_PRIVATE_CAR", "version": "2026.10", "effective_from": "2026-10-01",
  "jurisdictions": ["IN"],
  "terms_allowed": ["1Y", "1Y_OD_3Y_TP"],
  "line": {
    "pattern": "MotorPrivateCar",
    "coverables": ["Vehicle", "Driver"],
    "coverages": [
      { "code": "MOTOR_TP", "existence": "required" },
      { "code": "MOTOR_OD", "existence": "electable",
        "terms": [ { "code": "idv", "type": "direct", "min": 5000000 },
                   { "code": "vol_deductible", "type": "option",
                     "options": [0, 250000, 500000], "default": 0 } ] },
      { "code": "PA_OWNER_DRIVER", "existence": "required",
        "available_when": "ownerType == 'individual'" },
      { "code": "ZERO_DEP", "existence": "electable",
        "available_when": "vehicle.ageYears <= 5", "requires": ["MOTOR_OD"] },
      { "code": "ROADSIDE_ASSIST", "existence": "suggested", "requires": ["MOTOR_OD"] }
    ]
  }
}
```

Amounts in paise. Codes, values and availability rules are illustrative; real values come from the approved product filing.

Appendix D. Rate routine (example)

```
routine: MOTOR_OD_PREMIUM
steps:
  - basis:    idv                                      # Insured Declared Value
  - lookup:   base_rate  from OD_BASE_RATE  by [zone, engine_cc_band, vehicle_age_band]
  - multiply: basis * base_rate                        -> od_basic
  - lookup:   ncb_pct    from NCB_SCALE     by [claim_free_years]
  - multiply: od_basic * (1 - ncb_pct)                 -> od_after_ncb
  - lookup:   vd_disc    from VOL_DED_DISC  by [vol_deductible]
  - subtract: od_after_ncb - vd_disc                   -> od_net
  - minimum:  od_net, MIN_OD_PREMIUM
  - round:    nearest rupee
  - prorate:  slice_days / term_days
output: cost MOTOR_OD

# Tax is a separate cost item on every premium cost:
routine: GST
steps:
  - multiply: taxable_premium * rate_from(TAX_TABLE, [jurisdiction, line, date])
```

Table names and steps are illustrative. Each step writes a line to the rating worksheet.

Appendix E. Glossary

Term	Meaning
Bind	Accept the risk: the policy becomes legally effective
Endorsement	A mid-term change to a policy (here: a policy change job)
Exposure (claims)	One claimant against one coverage within a claim
FNOL	First notice of loss: the first report of a claim
IDV	Insured Declared Value: the sum insured for a vehicle in Indian motor insurance

Term	Meaning
NCB	No-claim bonus: a discount for claim-free years
Out of sequence	A change effective earlier than a change already bound
Preemption	When one open job must be rebased because another job on the same policy bound first
Pro rata, short rate	Refund methods on cancellation: by exact days, or by a table that keeps more premium
Producer code	The code of a selling agent or broker; controls visibility and commission
Reserve	Money set aside for the expected cost of a claim
Subrogation	Recovering a paid claim from the party who caused the loss
Written premium	The premium a transaction adds or removes, on its accounting date